About this PDF

Built only from: **Theory of Computation (Automata and Formal Languages)** — https://nptel.ac.in/courses/106104028

Licence: CC BY-SA 4.0. Accessed .

Covers 11 of 11 syllabus sub-topics of this unit.

Types of Grammar

Definition

- **Type 0**: Unrestricted Grammars (Recursively Enumerable Languages). Productions have the form alpha -> beta, where alpha contains at least one variable and beta is any string of variables and terminals. These languages are recognized by Turing machines.
- **Type 1**: Context-Sensitive Grammars (Context-Sensitive Languages). Productions have the form alpha A beta -> alpha gamma beta, where A is a non-terminal, alpha and beta are context strings, and gamma is a non-empty string. The length constraint |alpha A beta| <= |alpha gamma beta| holds. These languages are accepted by Linear Bounded Automata.
- **Type 2**: Context-Free Grammars (Context-Free Languages). Productions have the form A -> alpha, where A is a single non-terminal variable and alpha is any string of variables and terminals. These languages are accepted by Push Down Automata.
- **Type 3**: Regular Grammars (Regular Languages). Productions are restricted to right-linear form (A -> w B or A -> w) or left-linear form (A -> B w or A -> w), where A, B are variables and w is a string of terminals. These languages are recognized by Finite Automata.

Explanation

The hierarchy is based on how restrictive the production rules are. In Type 0 grammars, no restriction is imposed on the left-hand side, allowing any string containing at least one variable to be replaced by any string of terminals and variables. In Type 1 grammars the left-hand side must contain a single variable surrounded by a context, and the right-hand side must be at least as long as the left-hand side, ensuring that the length of the string does not decrease. In Type 2 grammars the left-hand side contains exactly one variable, making the derivation independent of context; this restriction allows the language to be parsed with a stack. In Type 3 grammars the right-hand side contains at most one variable and it must appear at one end of the string, which makes the language recognizable by a finite automaton. The more restrictions a grammar has, the smaller the class of languages it can generate.

Key points

- Four types of grammars are defined by the form of their production rules: unrestricted, context-sensitive, context-free, and regular.

- Type 0 grammars generate recursively enumerable languages and are recognized by Turing machines.
- Type 1 grammars generate context-sensitive languages and are accepted by linear bounded automata.
- Type 2 grammars generate context-free languages and are accepted by push down automata.
- Type 3 grammars generate regular languages and are recognized by finite automata.

Chomsky's hierarchy of languages

Definition

- **Type 0**: Unrestricted grammars (recursively enumerable languages) with productions of the form alpha → beta, where alpha contains at least one variable and beta is any string of variables and terminals. These languages are recognized by Turing machines.
- **Type 1**: Context-sensitive grammars (context-sensitive languages) with productions of the form alpha A beta → alpha gamma beta, where A is a non-terminal, alpha and beta are context strings, and gamma is a non-empty string; the length constraint |alpha A beta| <= |alpha gamma beta| holds. These languages are accepted by linear bounded automata.
- **Type 2**: Context-free grammars (context-free languages) with productions of the form A → alpha, where A is a single non-terminal and alpha is any string of variables and terminals. These languages are accepted by push-down automata.
- **Type 3**: Regular grammars (regular languages) with productions restricted to right-linear form (A → w B or A → w) or left-linear form (A → B w or A → w), where A, B are variables and w is a string of terminals. These languages are recognized by finite automata.

Explanation

The hierarchy reflects increasing restrictions on production rules, which in turn determine the computational power required to recognize the languages. As the constraints tighten from Type 0 to Type 3, the corresponding recognizers become less powerful: Turing machines for Type 0, linear bounded automata for Type 1, push-down automata for Type 2, and finite automata for Type 3. This ordering shows that every language in a lower type is also in all higher types, but not vice versa.

Key points

- The hierarchy consists of four types: unrestricted, context-sensitive, context-free, and regular.
- Each type imposes specific constraints on production rules and corresponds to a particular recognizer (Turing machine, LBA, PDA, FA).
- Context-free grammars generate languages accepted by push-down automata.

AcademicsGen — source: NPTEL — CC BY-SA 4.0

- Regular grammars generate languages recognized by finite automata.

Context-Free Grammar (CFG) and Languages

Definition

A Context-Free Grammar (CFG) generates context-free languages through recursive substitution of non-terminals using its production rules. The productions of a CFG have the form A -> alpha, where A is a single non-terminal variable and alpha is any string of variables and terminals. The language generated by a CFG is the set of all terminal strings that can be derived from the start symbol by repeated application of its production rules.

Explanation

Derivations are sequences of rule applications that start from the start symbol S and produce a terminal string. A leftmost derivation replaces the leftmost non-terminal at each step, while a rightmost derivation replaces the rightmost non-terminal. A parse tree is an ordered, rooted tree that represents the hierarchical structure of a derivation; the root is labeled with S, interior nodes with variables, and leaf nodes with terminals or epsilon. A CFG is ambiguous if some string in its language has two or more distinct leftmost derivations, rightmost derivations, or parse trees. Some languages are inherently ambiguous, meaning every grammar generating them is ambiguous.

Every context-free language accepted by a push-down automaton (PDA) can be generated by a CFG, and conversely, for any CFG there exists an equivalent PDA that accepts the same language. A PDA accepts a language either by reaching a final state with any stack contents or by emptying its stack, and both acceptance mechanisms define the same family of languages.

Key points

- A CFG consists of variables, terminals, production rules of the form A -> alpha, and a start symbol.
- Derivations, leftmost/rightmost derivations, and parse trees illustrate how strings are generated from the start symbol.
- Ambiguity arises when a string has multiple distinct derivations or parse trees; some languages are inherently ambiguous.
- Context-free languages are exactly the languages accepted by push-down automata, establishing equivalence between CFGs and PDAs.

Derivations and Parse trees

Derivations and parse trees are fundamental concepts in the study of context-free grammars, describing how strings are generated and represented.

Definition

A derivation is a sequential application of production rules starting from the start symbol S to generate a terminal string. A leftmost derivation (LMD) replaces the leftmost non-terminal at each step, while a rightmost derivation (RMD) replaces the rightmost non-terminal at each step. A parse tree (or derivation tree) is an ordered, rooted tree representing the hierarchical syntactic structure of a derivation. The root is labeled with the start symbol S, interior nodes are labeled with variables, leaf nodes are labeled with terminals or epsilon, and children of any node represent the right-hand side of a production rule applied to that variable. A CFG is called ambiguous if there exists at least one string in its language that has two or more distinct leftmost derivations, two or more distinct rightmost derivations, or two or more distinct parse trees.

Explanation

Derivations show the step-by-step substitution of non-terminals using production rules, while parse trees provide a visual representation of the hierarchical application of these rules, illustrating the structure of the generated string. Ambiguity arises when a string can be derived in multiple distinct ways, indicating that the grammar does not uniquely specify the syntactic structure of that string.

Key points

- A derivation is a sequence of rule applications from the start symbol to a terminal string.
- Leftmost and rightmost derivations differ in the order of non-terminal replacement.
- Parse trees depict the hierarchical structure of derivations with the start symbol at the root.
- Ambiguity occurs when a string has multiple distinct derivations or parse trees.

Ambiguity in grammars and languages

Definition

A CFG is called ambiguous if there exists at least one string in its language that has two or more distinct leftmost derivations, two or more distinct rightmost derivations, or two or more distinct parse trees.

Explanation

Ambiguity arises when a string can be derived in multiple ways, leading to different parse trees or derivation sequences. This indicates that the grammar does not uniquely specify the syntactic structure of that string.

Key points

- Ambiguity is identified by the existence of multiple distinct leftmost or rightmost derivations for a single string.
- It can also be detected through the presence of more than one parse tree for the same string.

- Some languages are inherently ambiguous, meaning every grammar generating them is ambiguous.

Push Down Automata (PDA): Definition

Definition

A Push Down Automaton (PDA) is a computational model consisting of a finite-state control, an input tape, and an auxiliary unbounded stack with last-in, first-out (LIFO) storage.

Formally, a non-deterministic PDA is defined as a 7-tuple M = (Q, Sigma, Gamma, delta, q0, Z0, F):

- Q is a finite set of states.
- Sigma is the finite input alphabet.
- Gamma is the finite stack alphabet.
- delta is the transition function mapping Q x (Sigma union {epsilon}) x Gamma to finite subsets of Q x Gamma*.
- q0 in Q is the initial state.
- Z0 in Gamma is the start stack symbol.
- F subset-of Q is the set of accepting final states.

The machine transitions depend on the current state, input symbol, and top stack symbol.

Moves

Definition

A move or transition of a PDA is determined by the current state, the current input symbol (or epsilon for an epsilon-move), and the top stack symbol. In making a move, the PDA changes state and replaces the top stack symbol with a string of stack symbols (push, pop, or no change).

Explanation

A move specifies how the PDA evolves from one configuration to another by consuming an input symbol (or none), updating its state, and modifying the stack according to the transition function.

Key points

- A move depends on the current state, input symbol (or epsilon), and top stack symbol.
- The transition changes the state and replaces the top stack symbol with a string of stack symbols.
- The replacement can represent pushing, popping, or leaving the stack unchanged.

Instantaneous descriptions

Definition

An Instantaneous Description (ID) represents the configuration of a PDA at any single moment. It is written as a triple (q, w, alpha), where q in Q is the current state, w in Sigma^* is the unread remaining input string, and alpha in Gamma^* represents the current stack contents with the leftmost symbol representing the top of the stack.

Explanation

The ID captures the entire instantaneous state of the PDA, including its control state, the remaining input, and the stack contents. Transitions between configurations are denoted by the turnstile relation, allowing the PDA to evolve from one ID to another as it processes input and manipulates the stack. A string is accepted by a PDA if there exists a sequence of IDs starting from (q_0, w, Z_0) and ending in an accepting configuration, either with a final state and empty input or with an empty stack, depending on the acceptance mechanism.

Key points

- An ID is a triple (q, w, alpha) that fully describes the PDA's state at a moment.
- The leftmost symbol of alpha is the top of the stack.
- Transitions between IDs are represented by the turnstile relation.
- Acceptance can be defined by reaching an ID with an empty input and a final state, or by reaching an ID with an empty stack.

Languages of pushdown automata

Definition

A language can be accepted by a PDA through two equivalent acceptance criteria:

- Acceptance by Final State: L(M)={ winSigma^* | (q_0,w,Z_0)|--^*(q,epsilon,alpha) with qin F and alphainGamma^* }
- Acceptance by Empty Stack (Null Stack): N(M)={ winSigma^* | (q_0,w,Z_0)|--^*(q,epsilon,epsilon) for some qin Q } Both mechanisms define exactly the same family of languages.

Explanation

Both acceptance mechanisms define exactly the same family of languages: for every PDA accepting by final state, there exists an equivalent PDA accepting by empty stack, and vice versa. Pushdown automata and context-free grammars accept and generate identical families of languages. For any context-free grammar G, an equivalent single-state PDA M can be constructed accepting by empty stack (CFG to PDA conversion). Conversely, given any PDA M accepting by empty stack, an equivalent CFG G can be constructed where composite variables of the form [p,X,q] represent paths between states that pop stack symbol X. A PDA is deterministic if for every state q, stack symbol X, and input symbol a, there is at most one move available, and if delta(q,epsilon,X) is non-empty, then delta(q,a,X) must be empty for all ainSigma. DPDAs accept deterministic context-free languages (DCFLs), which form a strict proper subclass of general context-free languages.

Key points

- A PDA accepts a language either by reaching a final state with any stack contents or by emptying the stack.
- The two acceptance criteria are equivalent; each PDA can be transformed into an equivalent PDA using the other criterion.
- Context-free grammars and pushdown automata accept the same family of languages.
- Deterministic pushdown automata accept deterministic context-free languages, a strict subset of all context-free languages.

Equivalence of pushdown automata and CFG-CFG to PDA-PDA to CFG

Definition

Pushdown automata and context-free grammars accept and generate identical families of languages. For any context-free grammar G, an equivalent single-state PDA M can be constructed accepting by empty stack. Conversely, given any PDA M accepting by empty stack, an equivalent CFG G can be constructed where composite variables of the form [p, X, q] represent paths between states that pop stack symbol X.

Explanation

The equivalence is established by two constructions. The first construction transforms a CFG into a PDA that accepts by empty stack. The second construction transforms a PDA that accepts by empty stack into a CFG using composite variables [p, X, q] to capture transitions that pop a stack symbol X while moving from state p to state q. These constructions show that the languages accepted by PDAs and generated by CFGs are the same.

Algorithm / Steps

1. CFG to PDA: Construct a single-state PDA M that accepts by empty stack.

2. PDA to CFG: For a PDA M accepting by empty stack, construct a CFG G whose variables are of the form [p, X, q] representing paths between states that pop stack symbol X.

Key points

- Pushdown automata and context-free grammars accept the same family of languages.
- A CFG can be converted into a single-state PDA that accepts by empty stack.
- A PDA accepting by empty stack can be converted into a CFG with composite variables [p, X, q].

Deterministic Pushdown Automata.

Definition

A deterministic pushdown automaton (DPDA) is a pushdown automaton in which, for every state q, stack symbol X, and input symbol a, there is at most one move available, and if a transition on epsilon with top stack symbol X exists, then no transition on any input symbol a is allowed with that same stack symbol.

Explanation

Because of this restriction, a DPDA has a single possible computation for each input string, which makes it deterministic. DPDAs accept deterministic context-free languages (DCFLs), which form a strict proper subclass of the general context-free languages.

Key points

- DPDA has at most one transition for each combination of state, input symbol (or epsilon), and top stack symbol.
- If a transition on epsilon exists for a given stack symbol, no transitions on any input symbol are allowed for that stack symbol.
- DPDA acceptance can be defined by final state or by empty stack, and both criteria accept the same family of languages.
- The languages accepted by DPDAs are deterministic context-free languages, a strict subset of all context-free languages.

Source and licence

This PDF is adapted from **Theory of Computation (Automata and Formal Languages)** (https://nptel.ac.in/courses/106104028), accessed .

Source licence: **CC BY-SA 4.0** — https://creativecommons.org/licenses/by-sa/4.0/

Changes made by AcademicsGen: the source text was rephrased, reorganised and grammar-corrected into study material; nothing added.

This PDF is shared under the same licence (CC BY-SA 4.0).